This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

SNAP-FPCIM: A 28-nm Embedded-Serial-Alignment and Non-2's-Complement-MAC Floating-Point Compute-in-Memory Macro With Adjustable Computation Precision

Xing Wang, *Graduate Student Member, IEEE*, Tianhui Jiao, *Graduate Student Member, IEEE*, Yucheng Du, Miaoyu Tang, Shaochen Li, Yi Yang, *Graduate Student Member, IEEE*, An Guo, Yuchen Tang, Zhichao Liu, Yuchen Ma, Yingxuan Zhou, Weiwei Shan, *Senior Member, IEEE*, Jun Yang, *Member, IEEE*, and Xin Si, *Member, IEEE*

***Abstract*—Digital compute-in-memory (DCIM), particularly floating-point CIM (FPCIM) has emerged as a promising technique to enhance energy efficiency with higher accuracy for artificial intelligence (AI) applications. However, previous works incurred substantial hardware overhead for FP multiplication and accumulation (MAC) and failed to leverage the bit-level sparsity across data formats. This work presents SNAP-FPCIM, an embedded-serial-alignment, non-2's-complement-MAC and adjustable computation precision FPCIM based on 6T static random-access memory (SRAM). The contributions of this work include: 1) a broadcast input and embedded light-convertor structure to enable BF16/INT8 MAC operations with improved input reusability; 2) an embedded area-efficient serial-alignment scheme with dual-bit-serial MAC; 3) a format-mixed N2CMAC flow to reduce circuit dynamic activity and signed computation overhead; 4) a mode-reconfigurable design to accommodate diverse computational precision requirements. A fabricated 28nm 64Kb SNAP-FPCIM macro supports FP-MAC operations of BF16 and INT8, achieving an energy efficiency of 62.84TFLOPS/W in BF16 mode and 90.15TOPS/W in INT8 mode.**

***Index Terms*—Artificial intelligence (AI), computing-in-memory (CIM), floating-point multiplication-and-accumulation (MAC), mixed-format computation.**

I. Introduction

THE explosion of artificial intelligence (AI) and machine learning (ML) technologies has driven a paradigm shift in computing architectures. Modern neural networks (NNs), such as convolutional neural networks (e.g., ResNet) and transformers (e.g., ViT), demand not only high computation throughput but also substantial memory bandwidth. As models grow in complexity, the traditional Von Neumann architecture, characterized by separate computation and memory units, faces the memory wall bottleneck, which limits both the energy efficiency and performance of the system. To address this problem, compute-in-memory (CIM) [1] has emerged as a promising architectural innovation that integrates computation units directly into memory arrays. This integration minimizes data movement, significantly improving energy efficiency, which is a key requirement for edge AI devices that operate under stringent power constraints.

CIM architectures can be implemented using various memory technologies, including volatile memories such as SRAM [2], [3] and eDRAM [4], [5], and non-volatile memories such as Flash [6], ReRAM [7], [8], and MRAM [9], [10], each offering different trade-offs in speed, density, and endurance. From the perspective of computation principles, CIM can be classified into analog domain (e.g., current, charge, and time domain) [11], [12], and digital domain [13], [14]. From the perspective of supported data types, CIM can be classified into integer (INT) CIM [13], [14] and floating-point (FP) CIM [7], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28]. INT-CIM employs fixed-point arithmetic with uniform bit-width scaling, whereas FPCIM utilizes FP representation comprising sign, exponent, and mantissa fields to achieve a broader dynamic range. Among these diverse CIM types, SRAM-based digital FPCIM has garnered increasing attention from researchers due to its high precision and strong robustness.

Some research on SRAM-based digital FPCIM designs focused on transforming FP data into fixed-point or integer formats and performing MAC computation with INT format. Guo et al. [16] proposed a 28nm 64Kb digital-domain floating-point computing unit (FCU) and double-bitcell (DBcell) CIM macro that reinterprets FP CNN workloads through optimized

Received 30 March 2026; revised 2 May 2026 and 31 May 2026; accepted 22 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 92464302, Grant 62522403, Grant 92264203, Grant 92464202, Grant 6232B2022, and Grant 62204036; in part by the National Science and Technology Major Project under Grant 2022ZD0118902, Grant 2023YFB4405100, and Grant 2023YFB4405102; in part by the Key Research and Development Program of Jiangsu Province under Grant BE2023020-1; and in part by the Fundamental Research Funds for the Central Universities. This article was recommended by Associate Editor J. Tian. *(Corresponding author: Xin Si.)*

The authors are with the School of Integrated Circuits, Southeast University, Nanjing 210096, China (e-mail: xinsi@seu.edu.cn).

Digital Object Identifier 10.1109/TCSI.2026.3706764

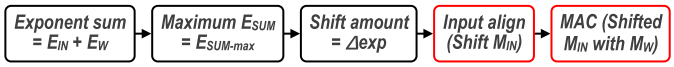
1549-8328 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Alignment scheme	Pre-alignment [15, 16]	Input-alignment [5, 19, 20]	Product-alignment [23, 26]
Align flow	Align M_{IN} and M_W to $E_{IN\text{-}max}$ and $E_{W\text{-}max}$ separately	Align M_{IN} based on E_{SUM} (E_{IN} + E_W)	Align M_{Prod} (M_{IN} × M_W) based on E_{SUM}
Computation precision	Low, separate alignment induces error	Middle, based on E_{SUM} but align M_{IN}	High, similar to CPU/GPU
Circuit overhead	Low, only extra peripheral alignment circuits	Middle, alignment circuits + low bit-width MAC	High, alignment circuits + high bit-width MAC

(a)



(b)

Fig. 1. Existing alignment schemes in FPCIM. (a) Comparison of alignment schemes. (b) Computation flow of input-alignment scheme.

fixed-point mappings. Tu et al. [15] presented ReCIM, a reconfigurable processor that unifies FP and INT pipelines via an alignment-free architecture and a novel booth multiplication scheme, effectively reducing the cycles while leveraging fixed-point representations for in-memory computation.

For higher computation precision, true FPCIM, which supports direct floating-point multiply-and-accumulate (MAC) operations within memory, has been proposed. Wu et al. [19] introduced an input-alignment-based hybrid-domain FP-IMC macro that computes both exponent and mantissa within the same SRAM-based array, combining time-domain, digital, and analog-voltage processing for high energy efficiency. Khwa et al. [5] proposed an INT/FP dual-mode CIM macro using a zone-based input processing scheme, a reused exponent adder tree, and a stationary gain-cell array to enhance energy and area efficiency for both INT8 and BF16 MAC operations. Yue et al. [23] proposed a product-alignment-based high-preciosn FPCIM for compound AI. Fig. 1 summarizes three alignment schemes and the input-alignment computation flow.

Existing digital-domain true FPCIM designs suffer from large area overhead caused by shift-and-align operations, making them difficult to integrate within memory arrays. This limitation compromises input reusability and area efficiency. Meanwhile, most existing digital CIM implementations employ 2's complement (2C) data representation, which introduces challenges including inefficient sparsity utilization and extra sign-bit computation overhead.

To address these limitations, we proposed a 28nm digital SNAP FPCIM macro supporting BF16 and INT8 MAC operations. It integrates a broadcast-alignment architecture, an adaptive serial alignment scheme, and a format-mixed non-2's complement MAC scheme.

The remainder of this paper is organized as follows: Section II analyzes the motivation, key design challenges and possible solutions; Section III introduces the triple-stacked FPCIM architecture; Section IV details the embedded serial-alignment (ESA) scheme and adjustable computation precision (ASP) scheme; Section V presents the format-mixed Non-2's-Complement-MAC (N2CMAC) scheme; Section VI provides performance and measurement results; Finally, Section VII concludes the work.

II. Motivation, Challenges and Proposed Solutions

To meet the high-precision demands of complex tasks, high-precision MAC is essential. However, previous FPCIM designs struggle to simultaneously achieve both high computation accuracy and macro-level input reuse, primarily due to the incompatibility of alignment operations with CIM architectures. This limitation motivates the design of a broadcast-input FPCIM architecture with embedded alignment. Additionally, implementing digital-domain floating-point/integer CIM presents several notable challenges: 1) performance degradation and increased area overhead caused by peripheral parallel-alignment schemes; 2) substantial dynamic energy consumption in digital MAC circuits resulting from the low bit-level sparsity of negative weights represented in two's complement (2C) format, along with the additional overhead of sign-bit computation; and 3) rigid and insufficient FP-MAC precision for precision-sensitive scenarios.

A. Motivation: Trade-off Between Precision and Input Reuse

Fig. 2a illustrates the MAC operations in mainstream neural networks. Each feature and weight vector has a dimension of input channel (ICH), and each feature vector is multiplied and accumulated with output channel (OCH) weight vectors. Consequently, many CIM works support input reuse at the macro level to achieve higher area efficiency [2], [11], [16].

In the Sharefloat FPCIM scheme shown in Fig. 2b, the BF16 input matrix determines a shared exponent (SE) by identifying the exponent corresponding to the maximum absolute value. The remaining mantissa components are then aligned accordingly. Similarly, the BF16 weight matrix adopts a weight SE for pre-alignment. The SRAM array stores the pre-aligned weight matrices, while the aligned inputs are fed into integer MAC units. The final results are reconstructed by combining the accumulated values with the shared exponent. This scheme reduces the overhead of MAC circuits while preserving input reusability at the macro level. However, its precision is limited by the pre-alignment computation scheme, which introduces significant computation errors.

Fig. 2c illustrates the input alignment paradigm. For each input-weight vector pair, this paradigm first computes E_{SUM} (the sum of the input and weight exponents) and identifies the maximum exponent E_{MAX}. The input mantissa components are then aligned to E_{MAX}. Compared with Sharefloat's separate alignment, the E_{SUM}-based input alignment approach does not align the input and weight separately; instead, it aligns based on their exponent sum, thus achieving higher precision. However, it introduces complex shifting circuits, which incur additional hardware overhead and reduce input reusability.

As illustrated in Fig. 2d, to balance the trade-off between computational precision and input reusability, this work proposes a compact "broadcast-input, embedded alignment" architecture. While this design shares the same principle as the input alignment scheme described earlier, the proposed circuit and architecture implementation enables simultaneous alignment and MAC operations through spatial parallelism,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

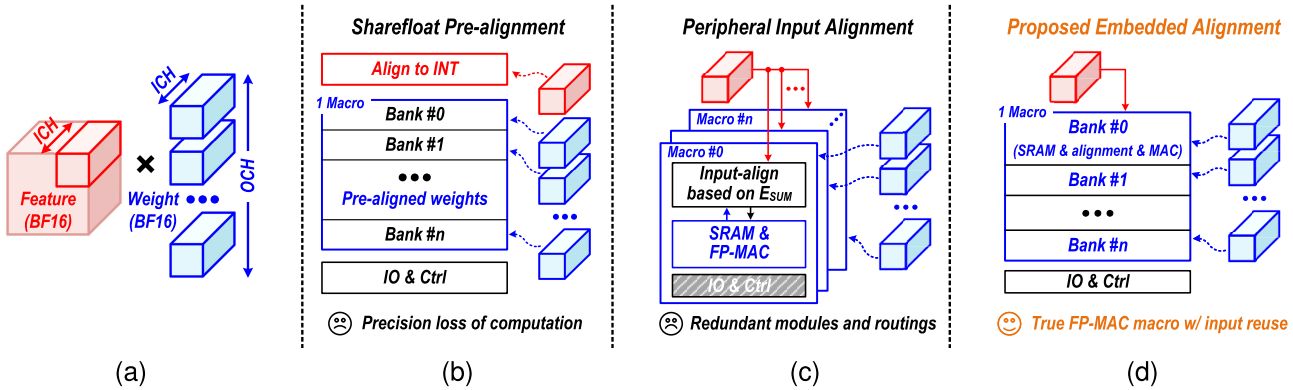


Fig. 2. Motivation of broadcast input and embedded light-convertor FPCIM structure. (a) Schematic diagram of the MAC operation. (b) Sharefloat scheme preserves input reusability but induces significant computation precision loss. (c) Peripheral E_{SUM}-based input alignment scheme maintains high computation precision but compromises input reusability. (d) Proposed compact "broadcast-input, embedded alignment" architecture.

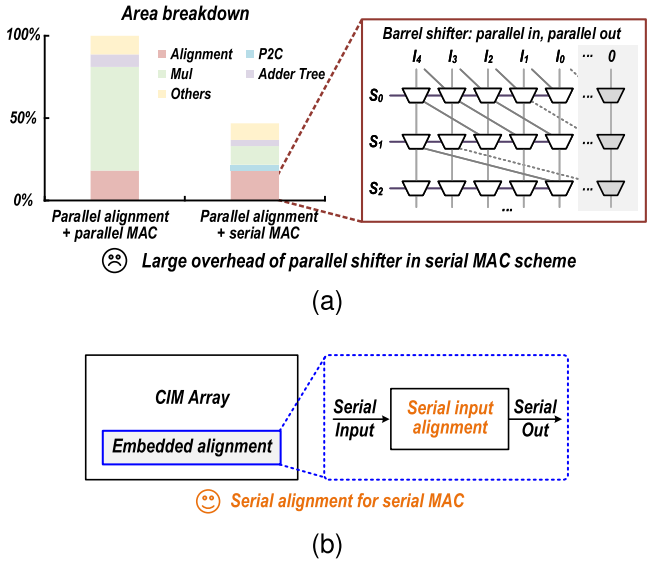


Fig. 3. Challenges of previous parallel-alignment schemes. (a) Large area overhead of peripheral parallel-alignment circuits in serial FP-MAC scheme. (b) Proposed serial alignment scheme for serial MAC.

thereby preserving computation accuracy while enhancing input reusability.

B. Challenge1: Large Overhead of Parallel-Alignment Circuit in Serial FP-MAC Schemes

Existing floating-point MAC designs can be classified by their alignment and computing dataflow into "parallel alignment, parallel MAC" [23] and "parallel alignment, serial MAC" [19]. Fig. 3a reports the area breakdown of these two schemes synthesized in 28 nm. In the "parallel alignment + parallel MAC" scheme, multipliers and adder trees perform MAC on 8-bit inputs and 8-bit weights entirely in parallel, which consumes 70% of the total area. The "parallel alignment + serial MAC" scheme reduces the MAC bit-width and thus cuts the MAC circuit cost significantly, but makes the alignment circuit the bottleneck (46% of the area). Alignment is typically realized with a barrel shifter. It operates in a "parallel-in, parallel-out" fashion, receiving all n input bits simultaneously and producing the shifted outputs in parallel. Because the wide dynamic range of FP formats yields large mantissa bit-width after shifting, a parallel MAC approach would require numerous multipliers and adders, leading to severe area overhead. While a serial MAC approach can reduce this overhead, it introduces extra parallel-to-serial conversion circuits. As a result, neither barrel shifters nor conventional parallel or serial MAC circuits suit area-efficient, regularly-structured CIM arrays, motivating the need for low-area floating-point MAC paradigms.

To overcome this, Fig. 3b proposes an embedded serial shift-and-align scheme. Unlike a barrel shifter, this scheme adopts a serial-in, serial-out dataflow that is naturally compatible with serial MAC operations. It reduces the logic overhead of mantissa alignment and improves area efficiency, making embedded alignment feasible for CIM.

C. Challenge2: Asymmetric Bit-Level Sparsity of 2C Format and Extra Sign-Bit Computation

Currently, most INT-CIM designs employ the 2's complement format. Compared to sign-magnitude representation, 2's complement allows direct MAC operations using unified multipliers and adders without requiring additional conversion steps. However, in neural network applications, the 2's complement format exhibits asymmetric sparsity issues that lead to significant computation overhead. As shown in Fig. 4a, the data distribution in ResNet50 approximately follows a zero-centered normal distribution, indicating the presence of numerous negative values near zero. These values demonstrate particularly low bit-level sparsity in 2's complement representation. For example, the 8-bit encoding of + 1 ("00000001") achieves 87.5% bit-level sparsity, while −1 ("11111111") shows 0% bit-level sparsity. Such low sparsity increases switching activity and consequently impacts circuit power consumption.

Existing solutions for signed MAC computation inevitably incur additional cycle counts and area overhead due to sign-bit processing requirements. Fig. 4b illustrates a serial MAC implementation where each cycle processes 2 bits of input, yet an 8-bit input requires 5 cycles to complete due to

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

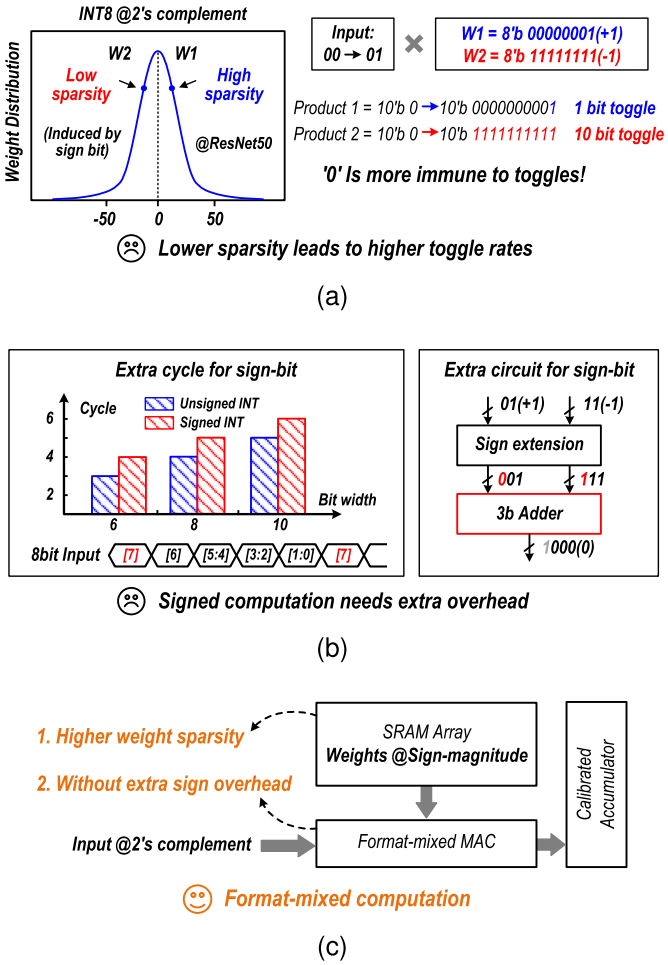


Fig. 4. Challenges of digital MAC circuits in 2C format. (a) Low sparsity of 2C negative weights. (b) Additional sign bit computation. (c) Proposed format-mixed MAC scheme.

separate sign-bit processing. Also, the adders require extra area overhead for sign-extension.

To address these challenges, this work proposes a format-mixed MAC scheme where inputs maintain the 2's complement format while weights adopt the sign-magnitude representation, as shown in Fig. 4c. This approach preserves weight sparsity while enabling transformation of signed MAC operations into unsigned MAC and a sign compensation through format adaptation, thereby eliminating sign-bit computation overhead.

D. Challenge3: Higher Computation Precision Requirements in Precision-Sensitive Scenarios

In neural network applications, different application scenarios impose varying demands on computational precision. For tasks that are highly sensitive to accuracy—such as autonomous driving, medical diagnosis, or other safety-critical applications—even minor computational errors can lead to incorrect predictions or system failures. These scenarios require extremely low error rates, making it essential to preserve sufficient precision throughout the computation. Prior works in FPCIM, such as [5], [19], and [20], typically preserve

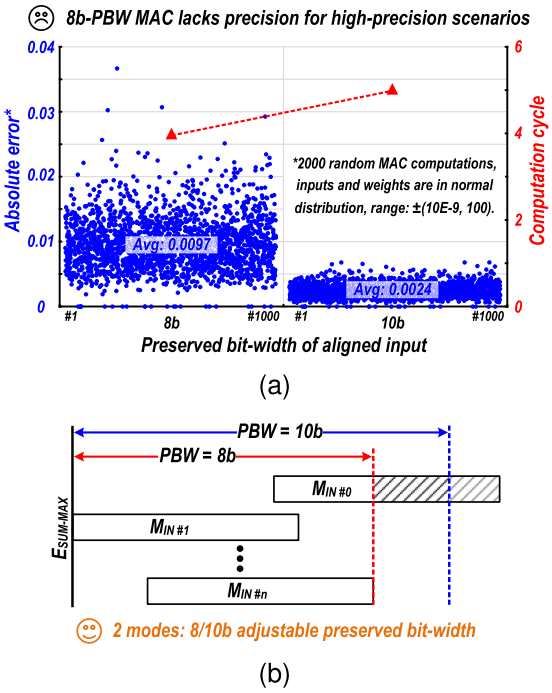


Fig. 5. Insufficient MAC precision for precision-sensitive scenarios. (a) Computation precision and cycle comparison between 8b/10b preserved bit-width. (b) Proposed adjustable preserved bit-width design.

8 bits after input alignment. However, the MAC precision offered by 8-bit preserved bit-width proves insufficient for such accuracy-critical inference tasks. Two representative tasks are evaluated to demonstrate this issue: ResNet-18 inference on ImageNet and VGG-16 training on CIFAR-10. For ResNet-18 inference, 8-bit preservation incurs a 0.020% accuracy drop compared to the full-precision baseline, while 10-bit preservation achieves 0.000% degradation, fully retaining baseline accuracy. For VGG-16 training, 8-bit preservation leads to unstable training dynamics and failure to converge, whereas 10-bit preservation successfully reaches 91.240% accuracy, matching the expected performance. These results underscore that 8-bit preserved width is inadequate for both inference and training in complex tasks, while 10-bit provides sufficient precision. To highlight this limitation further, Fig. 5a shows the computational error over 2000 random trials under a normal distribution. In contrast, preserving 10 bits increases computation cycles by only 25% compared to 8 bits, yet delivers a 4× reduction in error—making it significantly more suitable for precision-sensitive scenarios.

To address this challenge, as illustrated in Fig. 5b, this work introduces an adjustable preserved bit-width design that supports both 8b and 10b modes, allowing the system to flexibly meet varying application requirements without unnecessary overhead.

III. Proposed Triple-Stacked FPCIM Architecture and Configurations

A. "Alignment-Array-MAC" Structure

Fig. 6 presents the proposed "Alignment-Array-MAC" triple-stacked CIM architecture, integrating control modules,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

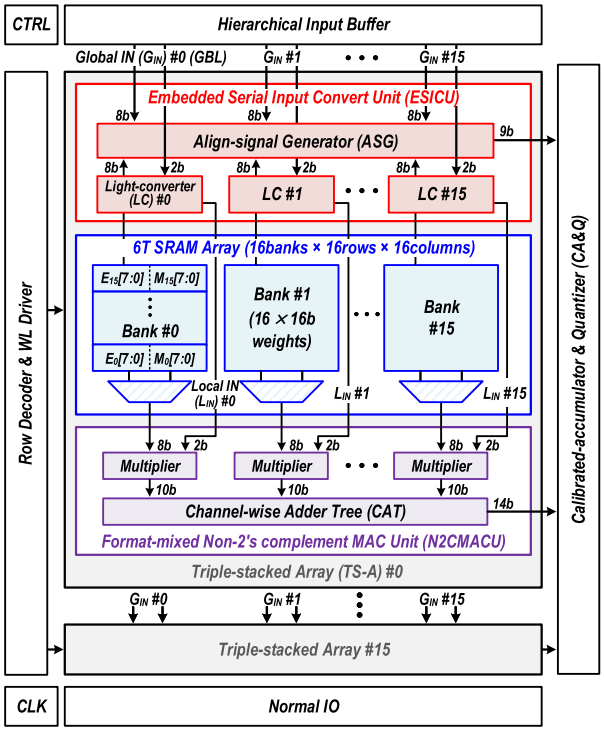


Fig. 6. Proposed "Alignment-Array-MAC" triple-stacked CIM architecture.

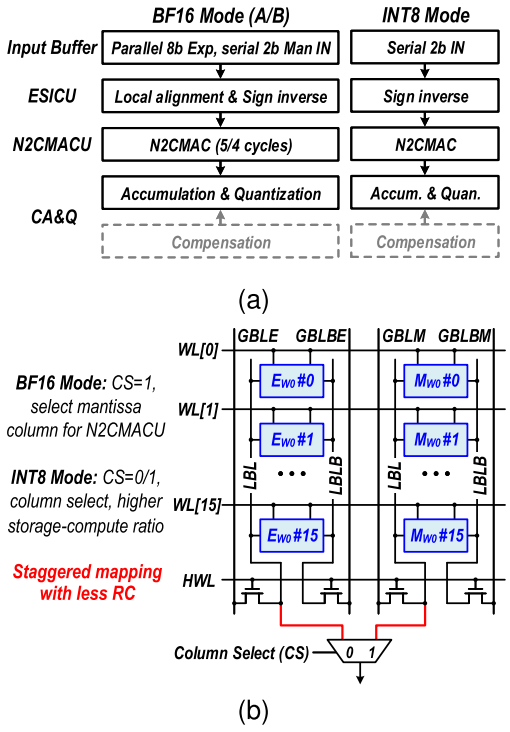


Fig. 7. Configurations of proposed SNAP-FPCIM. (a) BF16A/B and INT8 computation modes. (b) SRAM configurations in FP/INT modes.

hierarchical input buffer, row decoders & WL drivers, I/O module, and clock drivers in its peripheral circuitry. The central structure comprises 16 triple-stacked arrays (TS-A) sharing global bitlines (GBL) and global inputs (G_{IN}). Each TS-A embeds a 16-row ×256-column 6T SRAM array divided into 16 banks, where each bank contains a 16 × 16 SRAM bitcell array capable of storing 16 BF16 or 32 INT8 weights.

Above the memory array, an embedded serial input conversion unit (ESICU) processes input and weight exponents, generates mantissa shift control signals, and outputs exponent results of MAC. Beneath the array, a format-mixed non-2's complement MAC unit (N2CMACU) accomplishes MAC for aligned input and weight mantissas. To the right of the TS-As, a calibrated accumulator & quantizer (CA&Q) performs cyclic accumulation with compensation, ultimately producing normalized results in FP format.

The vertically compact triple-stacked layout minimizes area overhead while enabling scalability along the GBL direction. Expanding the TS-A count enables multiple TS-As to share identical inputs via GBLs, thereby optimizing the input reusability.

B. Mode Configuration

The proposed CIM macro supports 3 computation modes: BF16A, BF16B, and INT8. To enable efficient mode reconfiguration, the input buffer, SRAM array, and ESICU are functionally adaptive, as illustrated in Fig. 7a.

In BF16A/BF16B modes, inputs feature 8-bit parallel exponents and 2-bit serial mantissas. The ESICU aligns mantissa based on exponents, converts sign bits, and generates shift controls, while the N2CMACU executes 5/4-cycle serial MAC operations. The CA&Q accumulates, compensates and quantizes the results. Compared to BF16B, BF16A extends the computation by one cycle to preserve higher intermediate precision, reducing errors for high-accuracy applications. In INT8 mode, inputs are processed as 2-bit serial streams. The ESICU performs sign-bit conversion, the N2CMACU computes over 4 cycles, and the CA&Q processes to obtain final results.

Fig. 7b details the implementation of the 6T SRAM array: bitcells in each column connect via local bitlines (LBL/LBLB), which interface with global bitlines (GBL/GBLB) through transmission gates controlled by horizontal wordlines (HWL). During read/write operations, HWL activates the transmission gates to connect GBL/GBLB with LBL/LBLB. During computation, HWL isolates GBL/GBLB from LBL/LBLB.

To mitigate RC overhead from SRAM reconfiguration, weight exponents and mantissas are mapped to adjacent columns. A multiplexer (MUX) dynamically selects target columns: in BF16 modes, it selects mantissa columns to the N2CMACU; in INT8 mode, it flexibly accesses either exponent or mantissa columns, maximizing storage-compute ratio (SCR) through spatial reuse.

IV. Proposed Embedded Serial Alignment Scheme

The embedded serial alignment scheme consists of two components: the Alignment-Signal Generator (ASG) and the Light-Serial Converter (LC). The ASG generates shift control signals based on the exponents of inputs and weights in each cycle, controlling the LCs to perform serial shifting.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

A. Align-Signal Generator

The alignment-signal generator (ASG) comprises four functional modules: the exponent adder (EA), the maximum value finder (MVF), the shift destination tracer (SDT) and the equality comparator (EC).

Fig. 8a illustrates the ASG architecture and computation workflow. In the depicted example, the EA first computes the sums of 16 pairs of exponent inputs (E_{IN}) and exponent weights (E_W), generating sixteen 9-bit summation values ($SUM_{0\text{-}15}[8:0]$). For instance, $SUM_0[8:0]$ is 110101001, $SUM_1[8:0]$ is 110100111, and $SUM_{15}[8:0]$ is 110100100. The MVF then identifies the maximum value (MAX = 11010100) by comparing the upper 8 bits of $SUM_{0\text{-}15}[8:1]$, which serves as the initial value for the SDT. Subsequently, the SDT decrements its stored value by 1 in each cycle. Concurrently, the EC compares the current SDT value with the upper 8 bits of each SUM[8:1] every cycle. When a match is detected, the corresponding SHIFT control signal is set to high, thereby triggering serial shifting operations in the light-serial converter (LC).

Fig. 8b demonstrates this process through timing waveforms. At phase P_0, the SDT is initialized to 11010100, which matches $SUM_0[8:1]$, causing the $SHIFT_0$ signal to be activated and remain high until the reset signal occurs. During P_1, the SDT decrements to 11010011, matching $SUM_1[8:1]$, thereby setting $SHIFT_1$ to high. In P_2, the SDT further decrements to 11010010, aligning with $SUM_{15}[8:1]$ and activating $SHIFT_{15}$. This progression continues until phase P_4 (BF16A mode) or phase P_3 (BF16B mode), where a global reset signal clears all SHIFT control signals.

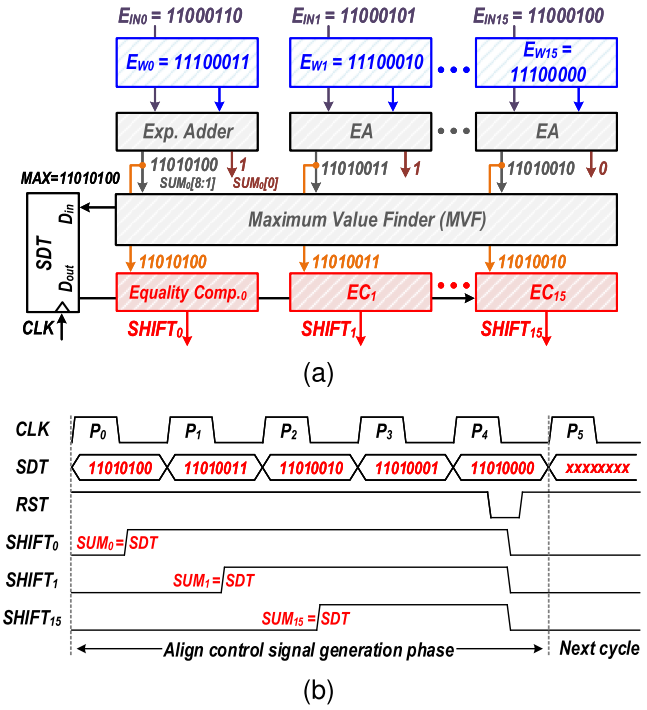


Fig. 8. Details of the proposed alignment-signal generator. (a) Overall structure. (b) Waveform of SDT and shift control signals (in BF16A mode).

B. Light-Converter

The light-converter (LC) consists of 3 modules: the self-alignment controller (SAC), the adaptive reset register chain (ARRC), and the output select & inverse unit (OSIU), as depicted in Fig. 9a.

The SAC module contains 4 registers that control the ARRC. When the SHIFT signal is low (indicating that mantissa shifting has not yet begun), the SAC activates its state to sequentially store the serial mantissa input (M_{IN}) in the ARRC. Upon the SHIFT signal transitioning to a high level, the SAC stops state transitions and the ARRC begins shifting out the stored mantissa per cycle, achieving serial alignment.

The adaptive reset register is an asynchronous set register with four input ports and one output port. When EN is low, the register operates in storage mode, capturing the input D at the rising edge of CLK and outputting it to Q. When EN is high, the register enters asynchronous set mode, directly propagating D to Q. The ARRC comprises seven interconnected registers with staggered connections, enabling a 2-bit shift per cycle during operation to minimize bit toggling and reduce computational latency. This seven-register configuration supports up to five computation cycles (three cycles to fill with 2-bit inputs and two cycles to shift them out), matching the 10-bit preserved width over five cycles in the BF16A mode. The OSIU selects the output bits based on the parity of the E_{SUM} and inverts the input sign bit. In integer mode, the ARRC and SAC are disabled, while the OSIU retains its sign-inverse functionality.

Fig. 9b illustrates waveforms of the example. When SHIFT signal is low, CP_1 is enabled, and the SAC state transitions from "1100" at phase P_0 to "0010" at P_1. Upon SHIFT rising to a high level, CP_2 is activated, freezing the SAC state at "0010" and triggering the ARRC state transitions to output stored data. At phase P_5, a system reset is initiated to prepare for subsequent computations.

C. Output Select & Inverse Unit

In the ARRC, data is shifted by 2 bits per cycle. Without the OSIU, the shift amounts would only be 0, 2, 4, etc., which cannot accommodate odd-numbered shift values. As shown in Fig. 9c, the OSIU addresses this by adding an extra sign-extension bit and utilizes a MUX to select the output bits, enabling shift amounts of 1, 3, 5, etc. The selection signal for the MUX is determined by the parity of the E_{SUM}. This design allows for serially shifting 2 bits per cycle, thereby increasing throughput, reducing toggle activity in the ARRC, and lowering computational power consumption.

D. Workflow of Serial-Alignment Scheme

Fig. 10 demonstrates the LC workflow and the alignment effect of the BF16A/B mode. At phase P_0 (SDT = 11010100), the 2b-G_{IN} consists of one sign bit and one mantissa bit, which are fed into the ARRC. Here, $ARRC_0$, $ARRC_1$, and $ARRC_{15}$ operate in storage mode, storing two copies of $G_{IN}[1]$ (sign bit) and one $G_{IN}[0]$ while outputting 2 bits based on OSIU. During P_1 (SDT = 11010011), $ARRC_0$ switches to shift mode, storing the current 2b-G_{IN0} while shifting out previously stored values,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

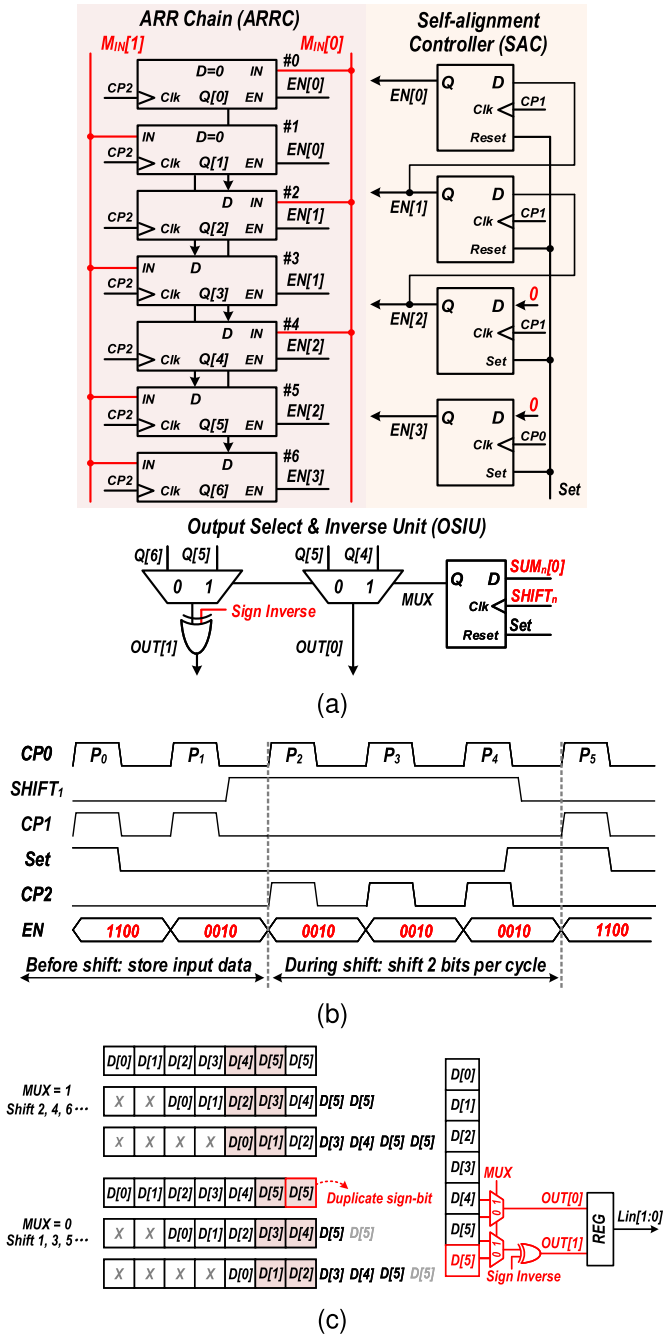


Fig. 9. Details of the proposed light-converter. (a) Circuit implementation. (b) Waveform of control signals. (c) Principles and details of the OSIU.

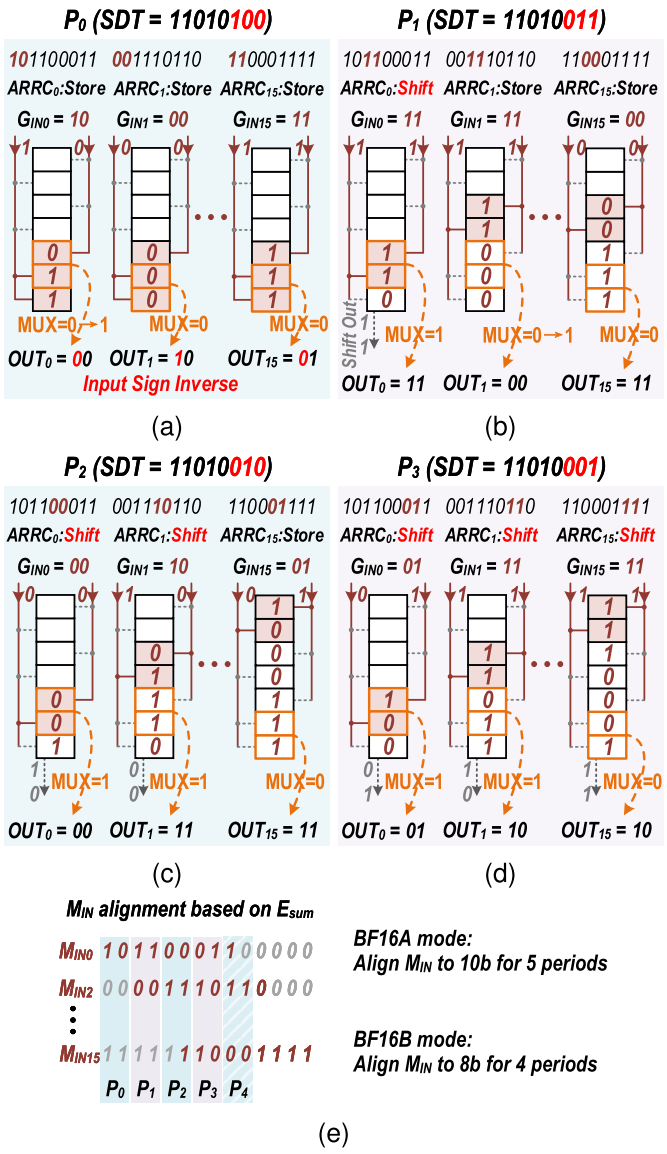


Fig. 10. Workflow example of the light-converter. (a) In P_0, all ARRCs are in storage mode. (b) In P_1, $ARRC_0$ switches to shift mode. (c) In P_2, $ARRC_1$ switches to shift mode. (d) In P_3, $ARRC_{15}$ switches to shift mode. (e) After 4/5 periods (BF16B/A mode) of serial alignment, the ARRC finishes the M_{IN} alignment based on E_{SUM}.

whereas $ARRC_1$ and $ARRC_{15}$ remain storing 2b $G_{IN1/15}$. At P_2 (SDT = 11010010), both $ARRC_0$ and $ARRC_1$ switch to shift mode for data output, while $ARRC_{15}$ continues buffering new inputs. By P_3 (SDT = 11010001), all three units ($ARRC_{0/1/15}$) are in shift mode, cyclically outputting stored values. Fig. 10e illustrates the serial shifting effect of the ARRC on $M_{IN0\text{-}15}$, which is equivalent to parallel shifting.

E. Support for Adjustable Preserved Bit-Width

Proposed alignment scheme transforms parallel shifting into serial workflow through global maximum-value search and cycle-wise comparison. By initiating ARRC shifting at different cycles based on Esum magnitudes, this design achieves the same effect as fully parallel shifting with parallel-to-serial conversion, significantly reducing circuit overhead. Additionally, the design supports two computation precision modes:

1) BF16A: Executes 5 computation cycles and preserves 10-bit shifted mantissa for precision-sensitive tasks;
2) BF16B: Executes 4 computation cycles and preserves 8-bit shifted mantissa for high-efficiency scenarios.

V. Proposed Format-Mixed N2CMAC Scheme

A. Principle of N2CMAC Scheme

Unlike the conventional 2's complement-based MAC scheme, the N2CMAC scheme stores weights in sign-magnitude format within the TS-As, while inputs are serially sent in 2's complement format. This scheme decomposes the format-mixed MAC operation into an unsigned MAC

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

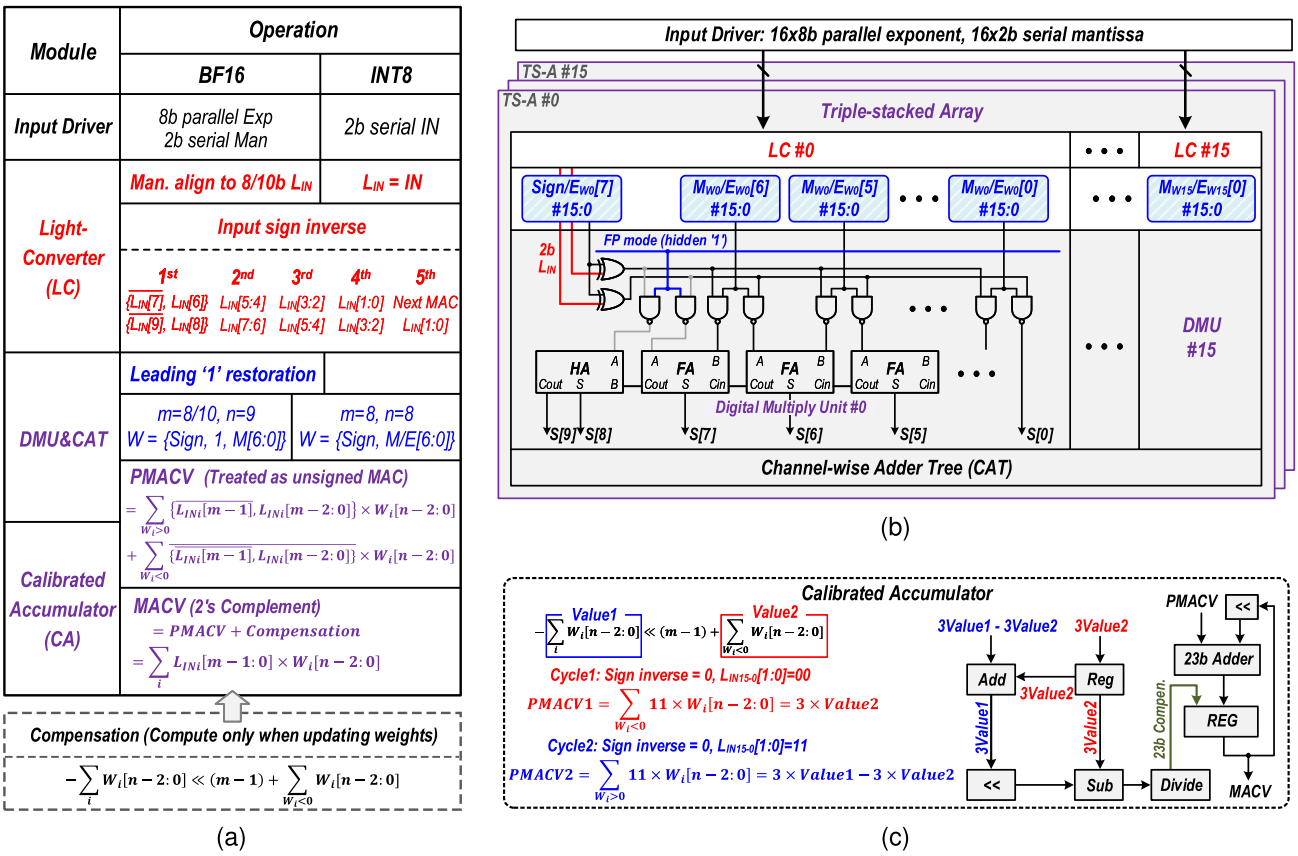


Fig. 11. Proposed Non-2's-Complement-MAC scheme. (a) BF16/INT8 Computation flow. (b) Implementation of digital multiply unit and channel-wise adder-tree. (c) Principle and implementation of calibrated accumulator.

component and a signed compensation term. The unsigned MAC reduces computation cycles and improves data sparsity, while compensation can be accomplished within the calibrated accumulator.

Detailed principles of the mixed-format MAC computation are elaborated in following equations. The MAC value for 2's complement format m-bit input IN and sign-magnitude format n-bit W can be expressed as Eq. 1.

$$MACV = \sum_{i} IN_i[m-1:0] \times W_i[n-1:0] \tag{1}$$

First, we transfer the sign bit of the sign-magnitude format to the input. If the weight is positive, the corresponding 2's complement input remains unchanged; if the weight is negative, the corresponding 2's complement input is inverted and incremented by 1. This transformation converts the weight into its absolute value, i.e., W[n-2:0], which can now be treated as an unsigned number. At this point, the multiply-accumulate operation can be expressed as Eq. 2.

$$\begin{aligned} MACV = &\sum_{W_i>0} IN_i[m-1:0] \times W_i[n-2:0] \\ &+ \sum_{W_i<0} (\overline{IN_i[m-1:0]} + 1) \times W_i[n-2:0] \end{aligned} \tag{2}$$

Next, we process the sign bit of the input. For an m-bit 2's complement input, adding a bias of 2^{m-1} allows it to be interpreted as an unsigned value. The MAC value can then be transformed into the following expression, where the final term compensates for the bias of 2^{m-1}.

$$\begin{aligned} MACV = &\sum_{W_i>0} (IN_i[m-1:0] + 2^{m-1}) \times W_i[n-2:0] \\ &+ \sum_{W_i<0} \left(\overline{IN_i[m-1:0]} + 1 + 2^{m-1}\right) \times W_i[n-2:0] \\ &- \sum_{i} W_i[n-2:0] \times 2^{m-1} \end{aligned} \tag{3}$$

Finally, after simplification, we obtain Eq. 4: the first two terms are treated as an unsigned MAC, while the last two terms serve as signed compensation, which is only dependent on weights and can be reused over multiple computation cycles.

$$\begin{aligned} MACV = &\sum_{W_i>0} \{\overline{IN_i[m-1]}, IN_i[m-2:0]\} \times W_i[n-2:0] \\ &+ \sum_{W_i<0} \overline{\{\overline{IN_i[m-1]}, IN_i[m-2:0]\}} \times W_i[n-2:0] \\ &- \sum_{i} W_i[n-2:0] \times 2^{m-1} + \sum_{W_i<0} W_i[n-2:0] \end{aligned} \tag{4}$$

B. N2CMAC Computation Flow and MAC Circuits

Fig. 11 depicts the N2CMAC computation flow and MAC circuits. As shown in Fig. 11a, the N2CMAC computation flow can be divided into four steps.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

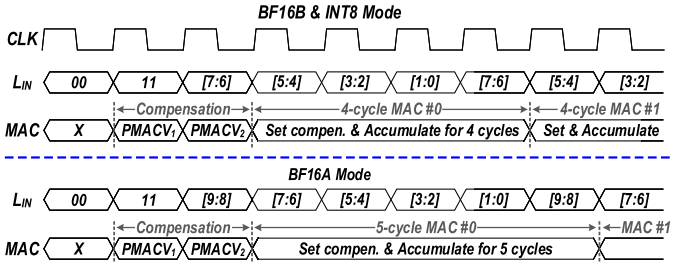


Fig. 12. The waveform of N2CMAC scheme for different modes.

Step 1: The 8-bit exponent for BF16A/B mode and the 2-bit serial input mantissa are provided by the input driver as the global inputs (G_{IN}), which are broadcast into multiple TS-As.

Step 2: In each TS-A, G_{IN} is converted into local input (L_{IN}) by the LC. In addition to performing the alignment function in the BF16A/B modes, the LCs also invert the sign bit of the two's complement input, completing the conversion of the input from a signed to an unsigned value. The converted L_{IN} is then sent to the subsequent MAC circuits for computation.

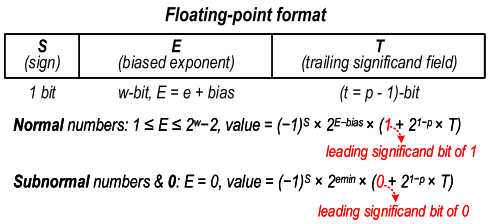
Step 3: In this step, the sign bits of sign-magnitude weights in the TS-A are utilized to determine whether to invert the L_{IN}, which are implemented with two XOR gates in the digital multiply unit (DMU), as shown in Fig. 11b. The products from the DMUs are accumulated through the adder tree, producing the partial MAC value (PMACV) for the current cycle. Thus, only one adder tree is needed in our design—no dual adder tree or routing circuit.

Step 4: The PMACV of each cycle is accumulated through the CA&Q, with the compensation value preloaded into the accumulator before the accumulation. In the BF16A/B and INT8 modes, the final MACV is obtained after 5/4, and 4 cycles of computation, resulting in 23-bit, 21-bit, and 20-bit MACVs, respectively.

Preloading sign-bit compensation into the CA requires two additional cycles, as illustrated in Fig. 11c. In the first cycle, L_{IN} = 00, sign inverse = 0. In the second cycle, L_{IN} = 11, sign inverse = 0. The results of these two cycles are combined to compute the weight-dependent sign-bit compensation, establishing the initial value of the accumulator. The waveforms of different computation modes are shown in Fig. 12.

C. Evaluation of the Leading Significand Bit Decoding

Fig. 13a illustrates the components of the floating-point format, which consists of three parts: sign, exponent, and mantissa. Depending on the value of the exponent, floating-point numbers can be categorized into normal numbers, subnormal numbers, and zero. For normal numbers, the leading significand bit is "1", while for subnormal numbers and zero, it is "0". To reduce circuit overhead, this work introduces a slight adjustment in the decoding process: the leading significand bit of BF16 weights is fixed to "1". To justify this approach, the proportion of subnormal weights was analyzed in models such as ResNet50, ResNet18, ViT, and YOLOv8, and the inference accuracy was compared with that of the software baseline. Experimental results demonstrate that the decoding method



(a)

Influence of omitting subnormal numbers in networks (BF16)

	Weight range (abs)	Subnormal ratio (%)	Accuracy (this work)	Baseline
ResNet50	7.58×10^{-13}-0.988	0	76.130%	76.140%
ResNet18	1.77×10^{-12}-1.016	0	69.770%	69.770%
ViT	9.71×10^{-10}-3.66	0	79.818%	79.834%
YOLOv8	0-17.49	0.000381	57.973%	57.974%

(b)

Fig. 13. Analysis of omitting subnormal numbers. (a) The definition of subnormal numbers. (b) Subnormal number ratio and the influence of omitting subnormal numbers.

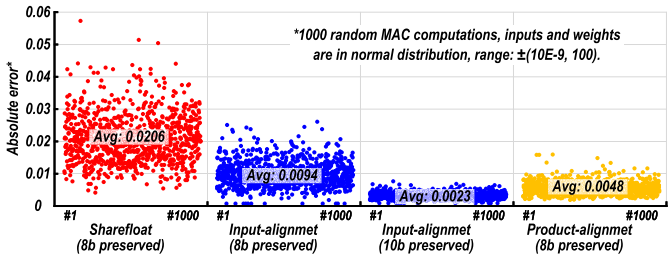


Fig. 14. Comparison of computation errors across sharefloat, input alignment and product alignment schemes.

employed in this work incurs almost no accuracy loss, as shown in Fig. 13b.

VI. Performance and Measurement Results

A. Performance of the Proposed Schemes

Fig. 14 shows the computation errors of three FP-MAC schemes: sharefloat, input alignment, and product alignment. With 8 bits preserved after alignment in all cases, we simulate 1000 random MAC operations. The results show that sharefloat has the largest computation error, followed by input-alignment, while product-alignment achieves the smallest error. If the input-alignment scheme preserves 10 bits after alignment, its computation error decreases significantly, demonstrating the impact of preserved bit-width on computation accuracy. This also indicates that BF16A (with 10 bits preserved) is more suitable for high-precision computing scenarios, whereas BF16B (with 8 bits preserved) is better suited for energy-efficient computing scenarios.

Fig. 15 presents the area and power comparisons of this work with previous approaches. By converting the parallel shift operations into a serial process and integrating this with serial MAC to form a pipelined architecture, the proposed serial alignment and serial MAC scheme achieves area and power reductions. Specifically, it demonstrates 3.83× and 1.56× reductions in area overhead, along with 1.71× and

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

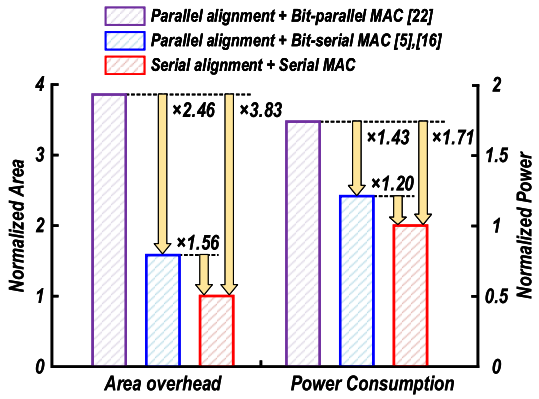


Fig. 15. Area overhead and power consumption comparisons across different alignment-MAC schemes.

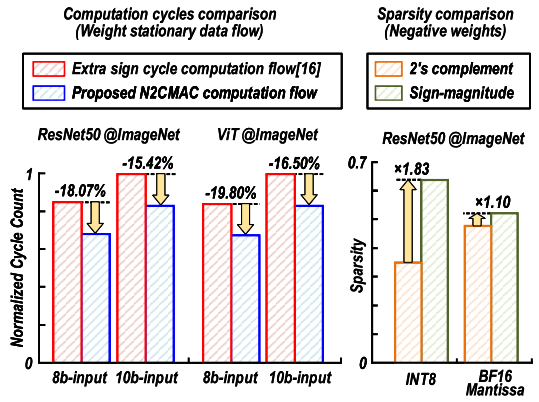


Fig. 16. Computation cycle and sparsity benefits of the proposed format-mixed N2CMAC computation flow.

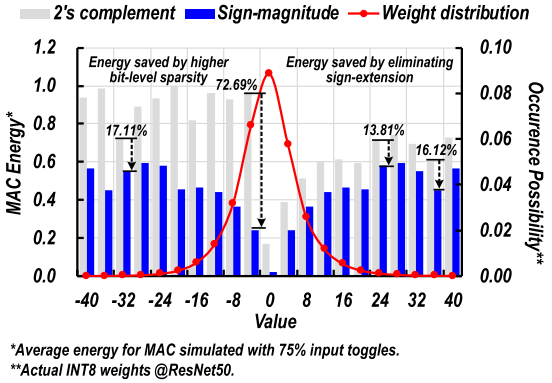


Fig. 17. Weight distribution of SM and 2C formats with corresponding MAC energy consumption.

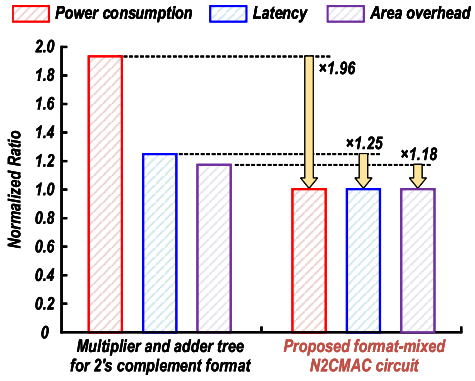


Fig. 18. Power, latency and area benefits of circuits for the proposed format-mixed N2CMAC scheme.

1.20× reductions in power consumption, compared to the previous parallel alignment with parallel MAC [22] and parallel alignment with serial MAC schemes [5], [16], respectively.

Fig. 16 demonstrates the performance gains achieved by the proposed N2CMAC computation flow. As shown in the chart, the flow reduces computing cycles across different networks with varying input precisions, achieving a 19.80% and 16.50% reduction for 8-bit and 10-bit inputs, respectively, tested on ViT (DeiT-S) with ImageNet. Furthermore, due to the use of sign-magnitude formatted weights, the sparsity of negative INT8 weights is significantly enhanced, resulting in a 1.83× improvement in ResNet50 compared to the conventional 2's complement format.

Fig. 17 presents the weight distribution and corresponding MAC energy consumption, illustrating the value-dependent energy efficiency of weight data in 2's complement and sign-magnitude format. It proves that computing with weight in sign-magnitude format could bring considerable energy reduction at both positive and negative zone by eliminating sign-extension and utilizing higher bit-level sparsity. While processing small negative number, which constitutes a large part of the total weight data, this feature can result in an energy efficiency improvement of up to 72.96%.

Fig. 18 shows the PPA improvement by the proposed format-mixed N2CMAC circuit. By adopting the sign-magnitude format, the weight sparsity is increased, reducing the logic-gate flip rate. Furthermore, the computation flow eliminates the need for sign-extension in adders. Collectively, these innovations result in 1.96×, 1.25×, and 1.18× lower power consumption, latency, and area overhead, respectively, compared to traditional 2's complement MAC circuits.

B. Measurement Results

This work implements a 64-Kb 6T SRAM-CIM test chip in 28-nm CMOS. Fig. 19 shows area/power breakdown. As in Fig. 19a, area is dominated by ESICU (29%), SRAM array (23%), and N2CMACU (19%), followed by IO (14%), WL driver (12%), controller (3%)–a balanced control/memory/compute allocation.

The power breakdown varies significantly across operating modes. In BF16 mode (Fig. 19b), ESICU consumes 53% (dominant in data scheduling/precision), N2CMACU 31%, SRAM 2%, controller 6%–indicating efficient memory access and low overhead. In INT8 mode (Fig. 19c), N2CMACU becomes primary at 60%, ESICU drops to 13%, SRAM 4%, controller 8%, IO negligible at 1%.

Fig. 20 shows area/energy efficiency under different voltages. Measurements use ResNet50/ViT on ImageNet with 80% input sparsity, 10% toggle rate, and real-case weights. Maximum MAC energy efficiency: 62.84 TFLOPS/W (BF16B), 46.55 TFLOPS/W (BF16A), 90.15 TOPS/W (INT8); area efficiency: 697.17 GFLOPS/mm^2, 553.63 GFLOPS/mm^2,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

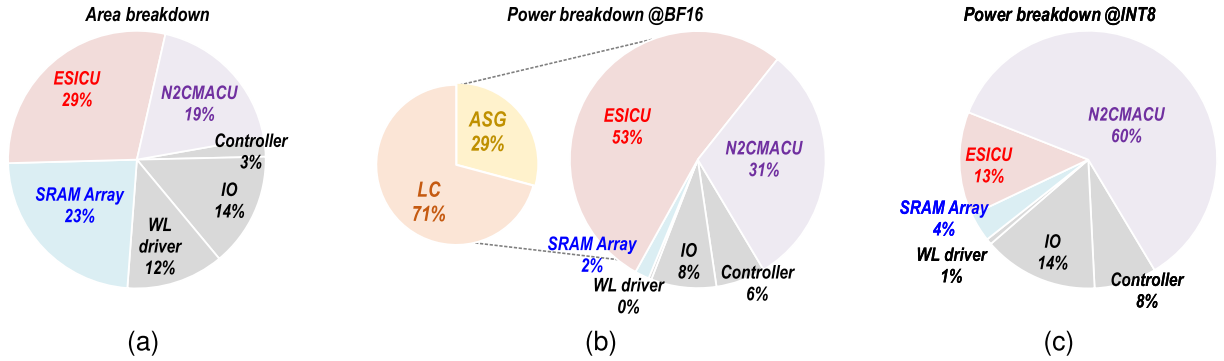


Fig. 19. Area and power breakdown. (a) Area breakdown. (b) Power breakdown in BF16A/B mode. (c) Power breakdown in INT8 mode.

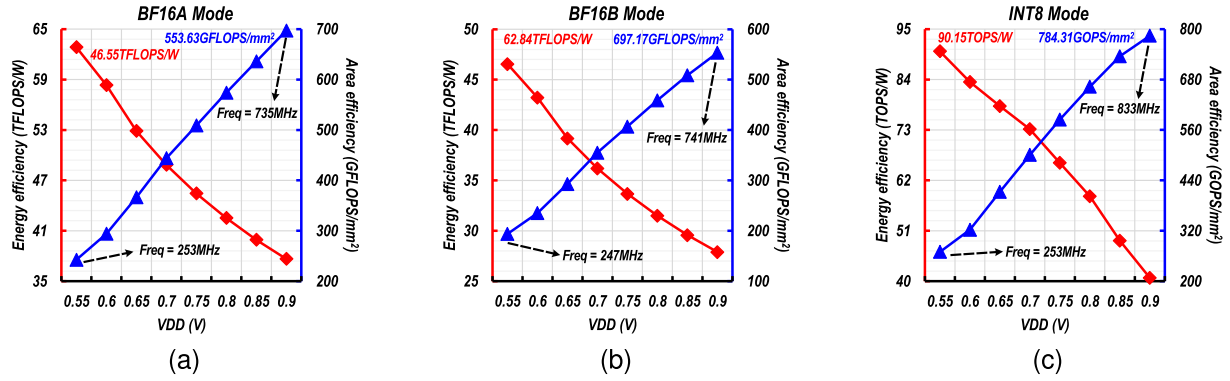


Fig. 20. Area and energy efficiency of SNAP-FPCIM, measured with real case: 80% input sparsity, 10% input toggle rate, real weights @ResNet50 & ViT.

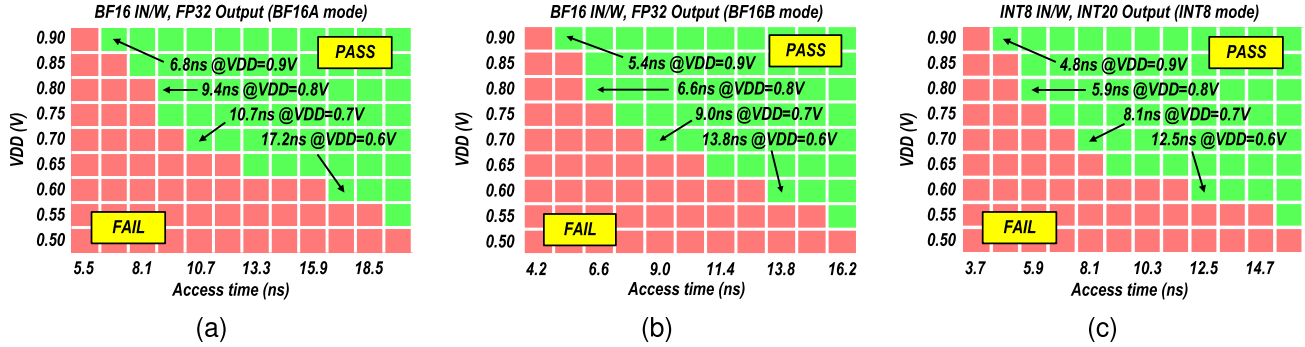


Fig. 21. Shmoo plots of SNAP-FPCIM. (a) BF16A mode (Access time = 5 clock cycles). (b) BF16B mode (Access time = 4 clock cycles). (c) INT8 mode (Access time = 4 clock cycles).

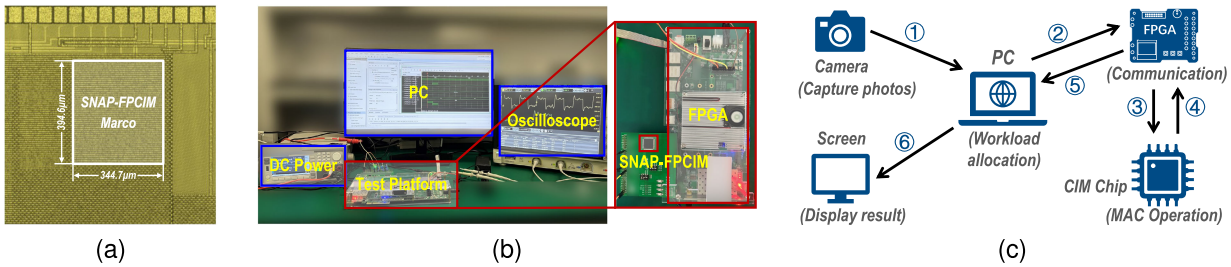


Fig. 22. Die photo and demo environment. (a) Die photo of SNAP-FPCIM. (b) Environment for chip test. (c) Demo system for a CNN-based vision model.

784.31 GOPS/mm^2. Fig. 21 shows measured Shmoo plots. At 0.9V, access times: BF16A 6.8ns (5 cycles), BF16B 5.4ns (4 cycles), INT8 4.8ns (4 cycles). BF16A/B have longer cycles due to hidden "1" computation lengthening the critical path.

C. Test Environment Setup

Fig. 22a shows the die micrograph of the proposed 28nm 64Kb SNAP-FPCIM macro. The macro occupies an area of 394.6μm $\times$344.7μm (0.136 mm^2) and supports BF16A,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE I
COMPARISON TABLE

	ISSCC'23 [16]	**ISSCC'24 [5]**		**ISSCC'25 [23]**	**ISSCC'25 [20]**		**This work**	
Memory capacity	64Kb	96Kb		224Kb	216Kb		**64Kb**	
Technology	28nm	16nm		28nm	16nm		**28nm**	
Cell structure	DBcell+FCU	Gaincell+DM-LCC		10T+9T	4T Gaincell+M2-IPU		**6T+ESICU&N2CMAC**	
Alignment scheme	Share-float	Input alignment		Product alignment	Input alignment		**Input alignment**	
Input precision (bit)	BF16	INT8	BF16	FP16	MXINT8	BF16	**INT8**	**BF16**
Weight precision (bit)	BF16	INT8	BF16	FP16	MXINT8	BF16	**INT8**	**BF16**
Output precision (bit)	BF16	INT23	FP32	FP32	FP32	FP32	**INT20**	**FP32**
Output-ratio[1]	0.3	1	1	1	1	1	**1**	**1**
Area (mm^2)	0.146	8[2]		2.67	8[2]		**0.136**	
Memory Density (Kb/mm^2)	438.36	12		83.9	27		**470.59**	
Norm. Area efficiency[3] ($GFLOPS/mm^2$)/($GOPS/mm^2$)	2567	2863	644	460-2440	2789	1407	**784.31**	**553.63 / 697.17**
Average Energy efficiency (TOPS/W)/(TFLOPS/W)	14.04-31.6	73.3-98.5	33.2-45.4	4.8-16.2	93.11 -133.51	63.14 -91.92	**25.47** **-90.15**	**17.83-62.84**
Energy efficiency test pattern	50% input sparsity	50-90% input sparsity		50-90% input bit-wise sparsity	–		25-80% input bit-wise sparsity	
Model	CNN	CNN		CNN/Transformer	CNN		**CNN**	**CNN/Transformer**
Inference Accuracy loss (ResNet18 @ImageNet)	-0.17% @CIFAR100	-0.580%	-0.220%	–	-0.39%	-0.02%	**-0.260%**	**-0.020 / 0.000%**
Inference Accuracy loss (ResNet50 @ImageNet)	–	–	–	-0.1%	–	–	**-0.110%**	**-0.010 / -0.010%**
Inference Accuracy loss (ViT @ImageNet)	–	–	–	-1.52%	–	–	–	**-0.028 / -0.016%**
Inference mAP50 loss (YOLOv8 @COCO)	–	–	–	–	–	–	**-1.006%**	**-0.004 / -0.001%**
Training Accuracy (VGG16 @CIFAR10)	–	–	–	–	–	–	–	**91.240%**[4]
FoM[5]	25.36	2.43	1.00	4.87	7.02	9.70	**17.56**	**34.60/43.57**

[1] Output-ratio = Real output resolution / Ideal output resolution.
[2] Estimated value based on the size of the die photo.
[3] Normalized to the same technology node (28nm), the scaling factor refers to [29].
[4] Trained in BF16A mode.
[5] FoM = IN-precision × W-precision × Output ratio × Memory density × Norm. area efficiency × Energy efficiency.

BF16B, and INT8 modes across 16 accumulation channels. During inference, this chip achieves accuracies of 76.130% / 76.130% / 76.030% on ResNet50 (@ImageNet), 79.818% / 79.806% on ViT (@ImageNet), and mAP50 of 57.973% / 57.970% / 56.968% on YOLOv8 (@COCO) under BF16A / BF16B / INT8 modes, respectively.

Fig. 22b illustrates the test system setup for this work. The system consists of a PC, an oscilloscope, a DC power supply, an FPGA, and a CIM Macro chip. Based on this test platform, the work implemented a YOLO v8-based demo, whose composition is shown in Fig. 22c.

Table I shows a comparison table of this work. Compared to prior SRAM CIM macros [5], [16], [20], [23], this work improves the FoM by 1.36–43.57×.

VII. CONCLUSION

This work proposes an SNAP-FPCIM macro that supports MAC computations for both INT8 and BF16 data formats, and incorporates input reusability at the macro level. The proposed "Alignment-Array-MAC" triple-stacked CIM architecture enables global input reuse while maintaining full floating-point precision, thereby reducing the overhead associated with input buffering and routing. An embedded area-efficient serial-alignment scheme with dual-bit-serial MAC is introduced, which serializes existing parallel shift-and-alignment schemes to be compatible with serial MAC circuits, effectively enhancing both energy efficiency and area efficiency. The proposed format-mixed N2CMAC flow employs the sign-magnitude format instead of 2's complement, increasing weight bit sparsity, reducing circuit activity, and lowering MAC power consumption. The "Preserved-bit Reconfigurable" scheme achieves FP-MAC with 2 accumulation precisions by adjusting computational cycles, catering to both inference and training application scenarios. An SNAP-FPCIM macro testchip, fabricated in a 28nm process with a 64Kb array, demonstrated an energy efficiency of 17.83-62.84 TFLOPS/W for BF16 operations in BF16B mode. This work achieves accuracies of 76.130% / 76.130% / 76.030% on ResNet50 (@ImageNet), 79.818% / 79.806% on ViT (@ImageNet), and mAP50 of 57.973% / 57.970% / 56.968% on YOLOv8 (@COCO) under BF16A / BF16B / INT8 modes, respectively.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

REFERENCES

[1] P. Chi et al., "PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Seoul, South Korea, Jun. 2016, pp. 27–39.

[2] X. Si et al., "15.5 A 28 nm 64Kb 6T SRAM computing-in-memory macro with 8b MAC operation for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 246–248.

[3] J.-W. Su et al., "15.2 A 28 nm 64Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 240–242.

[4] Y. He et al., "34.7 a 28nm 2.4Mb/mm^2 6.9–16.3TOPS/mm^2 eDRAM-LUT-based digital-computing-in-memory macro with in-memory encoding and refreshing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 578–580.

[5] W.-S. Khwa et al., "34.2 A 16 nm 96Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 568–570.

[6] L. Wang et al., "34.9 a flash-SRAM-ADC-fused plastic computing-in-memory macro for learning in neural networks in a standard 14nm FinFET process," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 582–584.

[7] T.-H. Wen et al., "34.8 A 22 nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 580–582.

[8] Z. Zhou et al., "ArPCIM: An arbitrary-precision analog computing-in-memory accelerator with unified INT/FP arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 5, pp. 2229–2242, May 2025.

[9] D.-Q. You et al., "14.1 A 22 nm 104.5TOPS/W μ-NMC-Δ-IMC heterogeneous STT-MRAM CIM macro for noise-tolerant Bayesian neural networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 1–3.

[10] H. Cai et al., "33.4 a 28 nm 2Mb STT-MRAM computing-in-memory macro with a refined bit-cell and 22.4–41.5TOPS/W for AI inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 500–502.

[11] B. Wang et al., "A 28nm horizontal-weight-shift and vertical-feature-shift-based separate-WL 6T-SRAM computation-in-memory unit-macro for edge depthwise neural-networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 134–136.

[12] S.-E. Hsieh et al., "7.6 A 70.85-86.27TOPS/W PVT-insensitive 8b word-wise ACIM with post-processing relaxation," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 136–138.

[13] H. Mori et al., "A 4 nm 6163-TOPS/W/b 4790 – TOPS/mm^2/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 132–134.

[14] H. Fujiwara et al., "34.4 A 3 nm, 32.5TOPS/W, 55.0TOPS/mm^2 and 3.78Mb/mm^2 fully-digital compute-in-memory macro supporting INT12 $\times$ INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 572–574.

[15] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.

[16] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 128–130.

[17] J. Shen et al., "A 12nm 4Mb 104.56-to-137.75TFLOPS/W charge-trap transistor-based computing-in-memory macro using analog-predict-digital-compute for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 69, San Francisco, CA, USA, Feb. 2026, pp. 514–516.

[18] Y. Yuan et al., "14.5 a 28 nm 192.3TFLOPS/W accurate/approximate dual-mode-transpose digital 6T-SRAM CIM macro for floating-point edge training and inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 258–260.

[19] P.-C. Wu et al., "A 22 nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2-70.2TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 126–128.

[20] W.-S. Khwa et al., "14.2 a 16 nm 216kb, 188.4TOPS/W and 133.5TFLOPS/W microscaling multi-mode gain-cell CIM macro edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 1–3.

[21] X. Wang et al., "14.3 a 28 nm 17.83-to-62.84TFLOPS/W broadcast-alignment floating-point CIM macro with non-two's-complement MAC for CNNs and transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 254–256.

[22] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, "FP-IMC: A 28nm all-digital configurable floating-point in-memory computing macro," in *Proc. ESSCIRC IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Lisbon, Portugal, Sep. 2023, pp. 405–408.

[23] Z. Yue et al., "14.4 A 51.6TFLOPs/W full-datapath CIM macro approaching sparsity bound and <2^{-30} loss for compound AI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 1–3.

[24] J.-C. Tien et al., "30.5 a 16 nm 72kb 120.5TFLOPS/W versatile-format dual-representation gain-cell CIM macro for general purpose AI tasks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2026, pp. 520–522.

[25] J. Yue et al., "A 28 nm 16.9-300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 1–3.

[26] X. Wang et al., "30.1 a 28 nm 127.54TFLOPS/W MXFP6 and 117.42TFLOPS/W MXFP8 compute-in-memory macro with adaptive-preserved-bit-width and serial-dual-bit-sliding schemes," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2026, pp. 512–514.

[27] C. Xue, Y. Yang, S. He, G. Du, Y. Wang, and Y. He, "A bit-partitioned floating-point 6T SRAM computing-in-memory macro based on dual-edge time-domain structure," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 12, pp. 7755–7768, Dec. 2025.

[28] C.-Y. Chang, C.-T. Huang, Y.-C. Chuang, K.-C. Chou, and A.-Y. Wu, "BFP-CIM: Runtime energy-accuracy scalable computing-in-memory-based DNN accelerator using dynamic block-floating-point arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 5, pp. 2079–2092, May 2024.

[29] S. Sarangi and B. Baas, "DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.



Xing Wang (Graduate Student Member, IEEE) received the B.S. degree in electronic science and engineering from Southeast University, Nanjing, China, in 2023, where he is currently pursuing the Ph.D. degree in integrated circuit engineering. His current research interests include floating-point computing in memory and AI accelerator.



Tianhui Jiao (Graduate Student Member, IEEE) received the B.S. degree from Southeast University, Nanjing, China, where she is currently pursuing the M.S. degree with the School of Integrated Circuits. Her research interests include SRAM design and computing-in-memory.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



Yucheng Du received the B.S. degree in microelectronics science and engineering from Southeast University, Nanjing, China, in 2025, where he is currently pursuing the M.S. degree in integrated circuit engineering. His research interests include computing in memory, AI accelerators, and memory design.



Miaoyu Tang is currently pursuing the bachelor's degree with the Chien-Shiung Wu College, Southeast University. His research interests include computing-in-memory circuit designs and AI accelerators.



Shaochen Li received the B.S. degree in electronic science and engineering from Southeast University, Nanjing, China, in 2024, where he is currently pursuing the M.S. degree. His research interests include software and hardware collaboration, memory, and computing in memory circuit designs.



Yi Yang (Graduate Student Member, IEEE) received the B.S. degree in information science and engineering from Southeast University, Nanjing, China, in 2024, where he is currently pursuing the M.S. degree with the School of Integrated Circuits. His research interests focus on digital circuit design and computing-in-memory architectures.



An Guo received the B.S. and M.S. degrees in electronic science and engineering and microelectronics science and engineering from Southeast University, Nanjing, China, in 2020 and 2022, respectively, where he is currently pursuing the Ph.D. degree in integrated circuit engineering. His research interests include memory, AI accelerators, and computing in memory.



Yuchen Tang received the B.S. degree from Suzhou University of Science and Technology, Suzhou, China, in 2022. He is currently pursuing the Ph.D. degree with the School of Integrated Circuit Science and Engineering, Southeast University, Nanjing. His research interests include computing-in-memory circuit and LLM accelerator design.



Zhichao Liu received the B.S. degree in electronic science and engineering from Southeast University, Nanjing, China, in 2023, where he is currently pursuing the Ph.D. degree in integrated circuit engineering. His current research interests include data stream, AI accelerator, and computing in memory.



Yuchen Ma received the B.S. degree in electronic science and engineering from the School of Electronic Science and Engineering, Southeast University, Nanjing, China, where he is currently pursuing the M.S. degree with the School of Integrated Circuits. His research interests include computing-in-memory and ASIC design.



Yingxuan Zhou received the B.S. degree in microelectronics science and engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2025. She is currently pursuing the M.S. degree in integrated circuit engineering with Southeast University, Nanjing, China. Her current research interests include AI accelerator and computing in memory.



Weiwei Shan (Senior Member, IEEE) received the Ph.D. degree in microelectronics from Tsinghua University, Beijing, China, in 2009. She is currently a Professor with the National ASIC Center, Southeast University, Nanjing, China. She has authored or co-authored more than 50 technical papers. Her research mainly focuses on variation resilient, adaptive VLSI circuits, and ultra-low-power SoC design.



Jun Yang (Member, IEEE) received the B.S. and Ph.D. degrees in electronic engineering from Southeast University, Nanjing, China, in 1999 and 2004, respectively. He is currently a Professor with the National ASIC System Engineering Research Center, Southeast University. He has authored and co-authored over 50 technical articles in conferences and journals. His researches focus on SRAM design, in-memory computing, and near-threshold design.



Xin Si (Member, IEEE) received the B.S. and Ph.D. degrees from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2016 and 2020, respectively. Since 2021, he has been an Associate Professor with the School of Integrated Circuits, Southeast University (SEU), Nanjing, China. He has an extensive publication record, including over 30 conference/journal articles. His research interests encompass memory, computing-in memory circuits, and AI chip designs.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:28:44 UTC from IEEE Xplore. Restrictions apply.